

DataFlow: LLM 时代统一数据准备与 workflow 自动化框架

DataFlow: An LLM-Driven Framework for Unified Data Preparation and Workflow Automation in the Era of Data-Centric AI

作者: Hao Liang 等

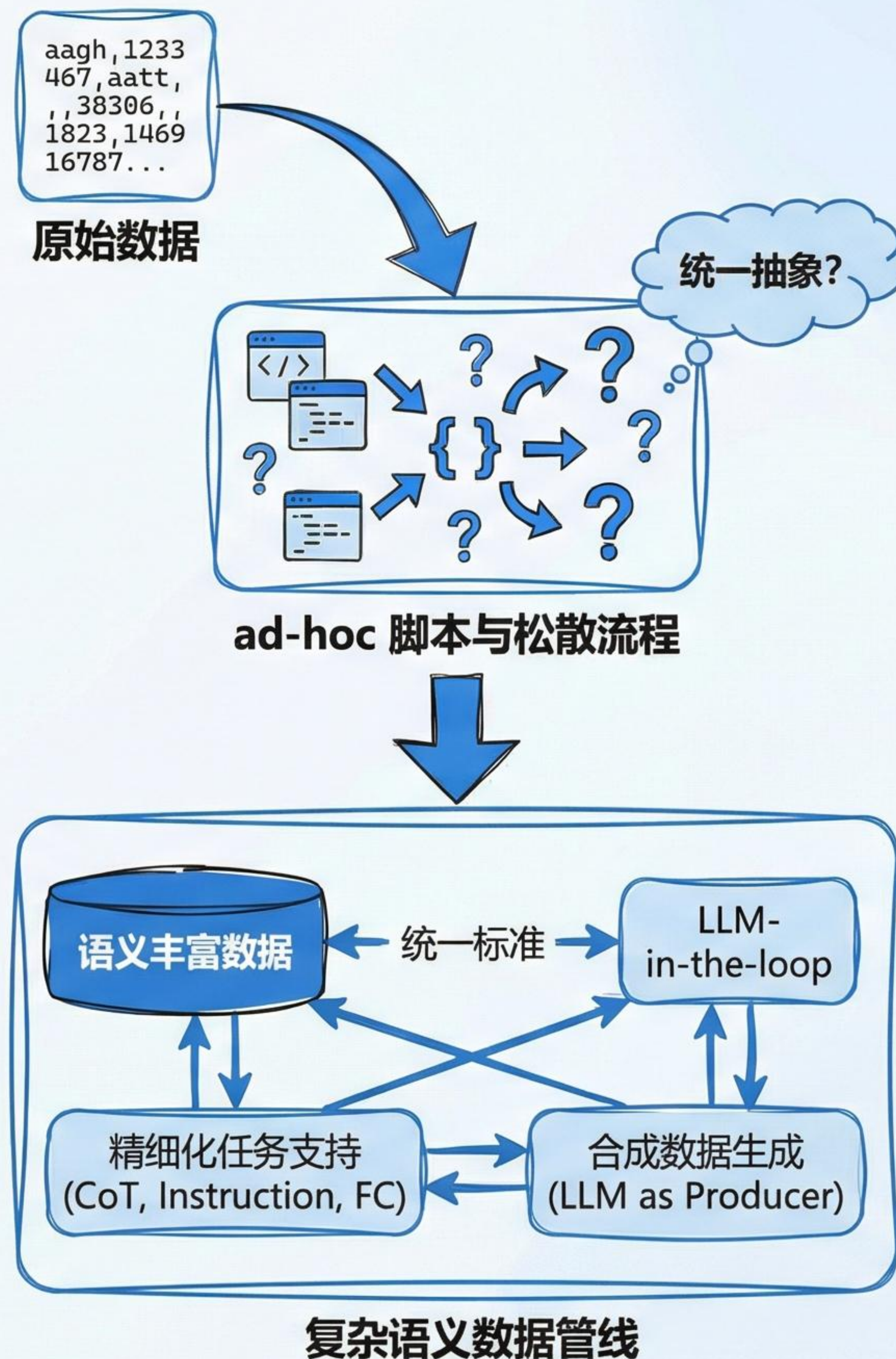
单位: Peking University 等

汇报人: XXX

日期: 2023年10月27日

问题背景与研究动机

- LLM 性能高度依赖高质量、大规模且语义丰富的数据。
- 当前数据准备仍以 ad-hoc 脚本和松散流程为主，缺乏统一抽象与标准。
- 传统大数据 ETL 系统（Spark、Dask、Hadoop）对语义密集、LLM-in-the-loop 的处理支持有限。
- 精细化后训练任务（CoT、Instruction、Function Calling）进一步放大数据准备质量的重要性。
- LLM 逐渐从数据“消费者”转变为数据“生产者”，合成数据 workflow 变得核心。



现有系统的局限与 DataFlow 的定位

现有系统的局限（如 NeMo Curator, Data-Juicer）

- 主要面向大规模抓取、过滤与清洗，缺乏对多步生成与语义精炼的原生支持。
- 以配置为中心（config-based）的设计对复杂、可调试的生成 workflow 支持不足。
- 对迭代式 model-in-the-loop 合成、评估、过滤和精修 workflow 抽象不够。

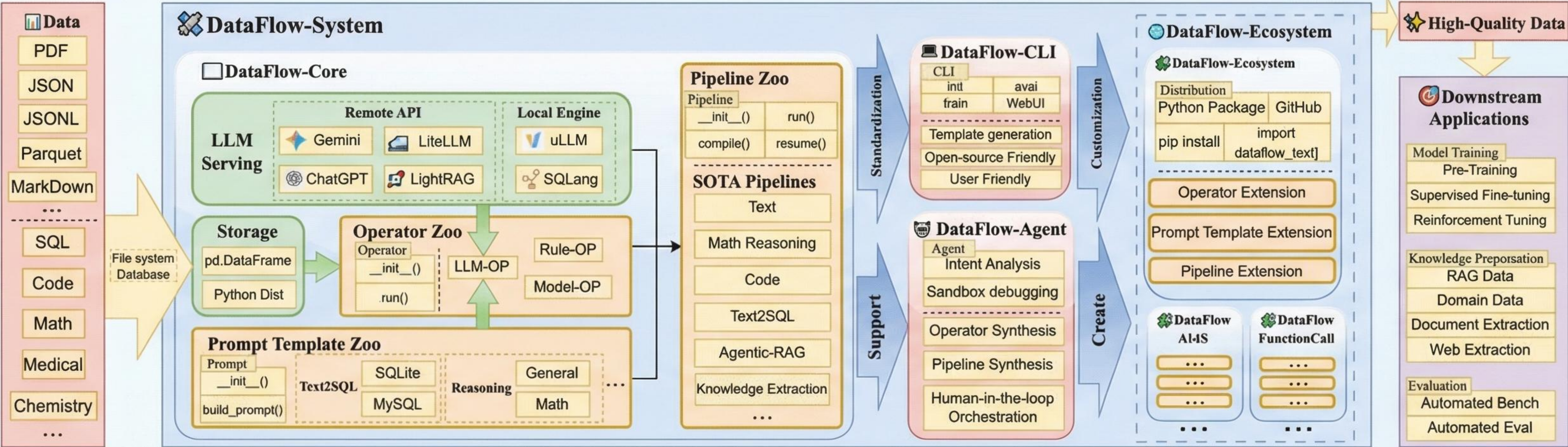
DataFlow 的定位与优势

- 将 LLM 驱动的数据合成提升为一等公民抽象，面向端到端数据准备。
- 目标：提供统一、可编程、可复用且可优化的 LLM 数据准备 workflow 基础设施。

Table 1: Placeholder table - the paper PDF was not provided, so the actual caption and data could not be extracted.

Column 1	Column 2	Column 3
Data 1	Data 2	Data 3
Data 4	Data 5	Data 6
Data 7	Data 8	Data 9

DataFlow 的整体系统结构



- **核心执行引擎:** 负责统一存储、运算符 (operators)、提示模板和LLM Serving。
- **Pipeline Zoo:** 为 text、math、code、Text-to-SQL、Agentic RAG、知识抽取等任务提供可复用管线。
- **用户控制层:** 包括 CLI (脚本化执行) 和 DataFlow-Agent (自然语言驱动的自动编排)。
- **扩展生态:** 通过 DataFlow-Extensions 以 Python 包形式发布领域专用 operators 与 pipelines。

DataFlow 的输出: 生成高质量、任务对齐的数据集，直接服务于下游 LLM 的训练与应用。

设计目标与核心理念

6大设计目标

- ✓ **易用性**: PyTorch 风格、IDE 友好接口, 便于调试复杂数据准备管线。
- ✓ **可扩展性**: 类似 `torch.nn.Module` 的模块抽象, 新算子与算法可即插即用。
- ✓ **统一范式**: 通过标准化抽象统一异构 workflow, 又保持跨领域定制能力。
- ✓ **性能效率**: 官方管线在多任务上达到或超越 SOTA 数据准备方法。
- ✓ **智能自动化**: 轻量级 agent 子系统支持自然语言意图下的管线构建与调整。
- ✓ **开源范式**: 对后端与管线的可复现分享, 为社区形成标准协议与生态。

核心理念演进关系



框架架构：全局存储与运算符交互

DataFlowStorage 抽象与读/写模式

- 统一表格化存储 (Tabular Representation)：面向指令、回复、CoT、分数与元数据等键值对。
- DataFlowStorage 提供后端无关 (backend-agnostic) 的 read() / write(data) 接口，屏蔽具体存储实现。
- 解耦设计：可替换为分布式/DB 后端而无需改动算子逻辑，实现扩展与优化。
- 默认实现基于 Pandas，支持 JSON/JSONL/CSV/Parquet 等常见格式。

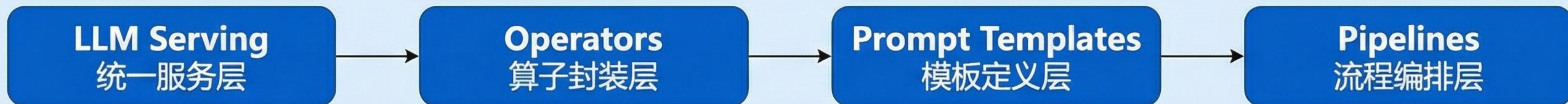
运算符 run() 读-变换-写模式图示

```
def run(self, storage: DataFlowStorage, **kwargs):  
    inputs = storage.read()           # 1. Read input  
    results = operator_transform(inputs, **kwargs) # 2. Transform the data  
    storage.write(results)            # 3. Write output
```

“读-变换-写”范式

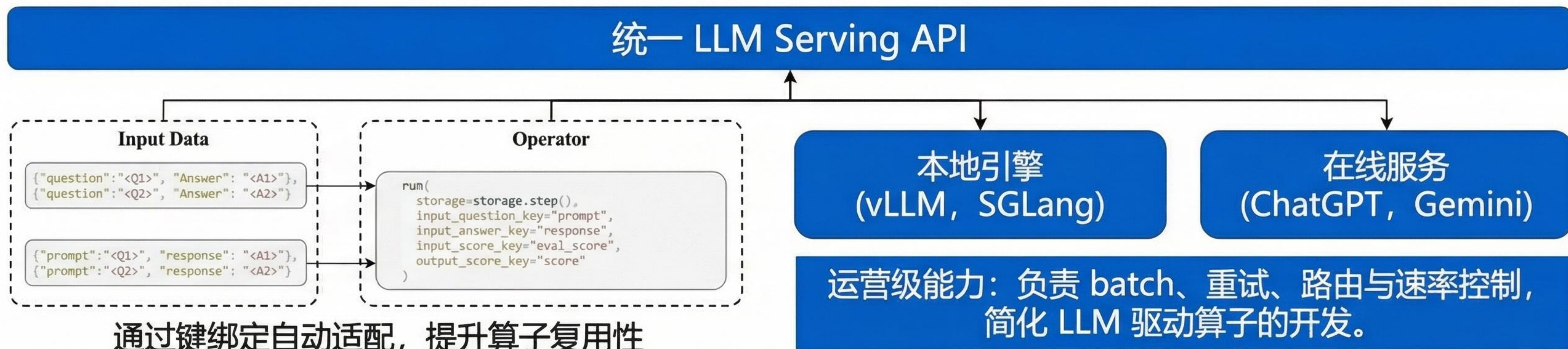


分层编程接口与 LLM Serving（框架架构）



- 分层接口：LLM Serving → Operators → Prompt Templates → Pipelines。
- 统一 LLM Serving API: `generate_from_input(user_inputs, system_prompt, json_schema)`。
- 屏蔽后端差异：支持本地引擎（vLLM、SGLang）与在线服务（ChatGPT、Gemini）。

统一 LLM Serving API 与后端交互示意图



算子与管线生态：Operator Zoo 与 Pipeline Zoo

算子库 (Operator Zoo)

近 200 个可复用运算符

- 四大分类：

-  生成 (generation)

-  评估 (evaluation)

-  过滤 (filtering)

-  精修 (refinement)

90+ 可复用 Prompt Template

支持一致的提示构造与行为规范。

管线库 (Pipeline Zoo)

统一 workflow 范式

“生成 - 评估 - 过滤 - 精修”

Pipeline Composition 接口

以 PyTorch Module 式 API 组合与复用，
无需任务特定 glue 代码。

六大 SOTA 模板管线



文本



Text-to-SQL



数学推理



Agentic RAG



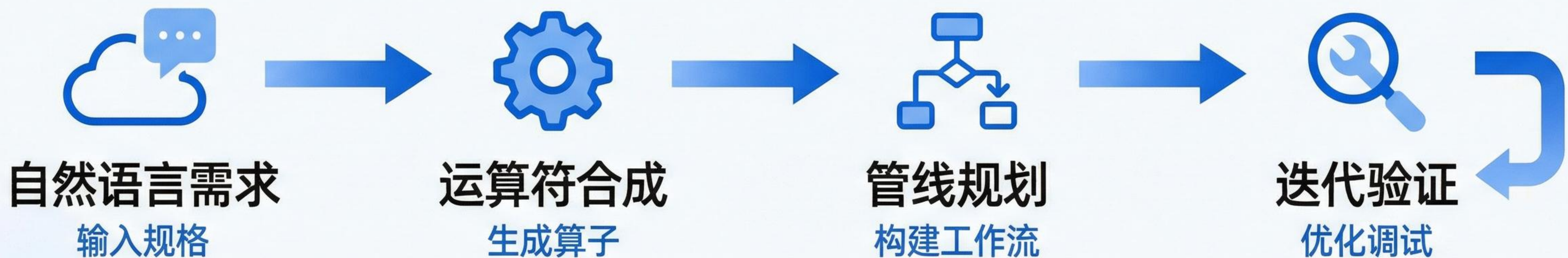
代码



知识抽取

DataFlow-Agent: 基于智能体的自动化 workflow 构建

- 智能编排层，将自然语言规格转化为可执行管线
- 核心功能：运算符综合、管线规划、迭代验证与调试
- 自动推荐与调整 Pipeline，降低手工工程与配置成本
- 基于统一抽象，与 CLI、核心算子库集成，可扩展为通用数据智能体



实验设置：多场景 LLM 数据准备评估

6 大数据准备场景与统一多领域设置



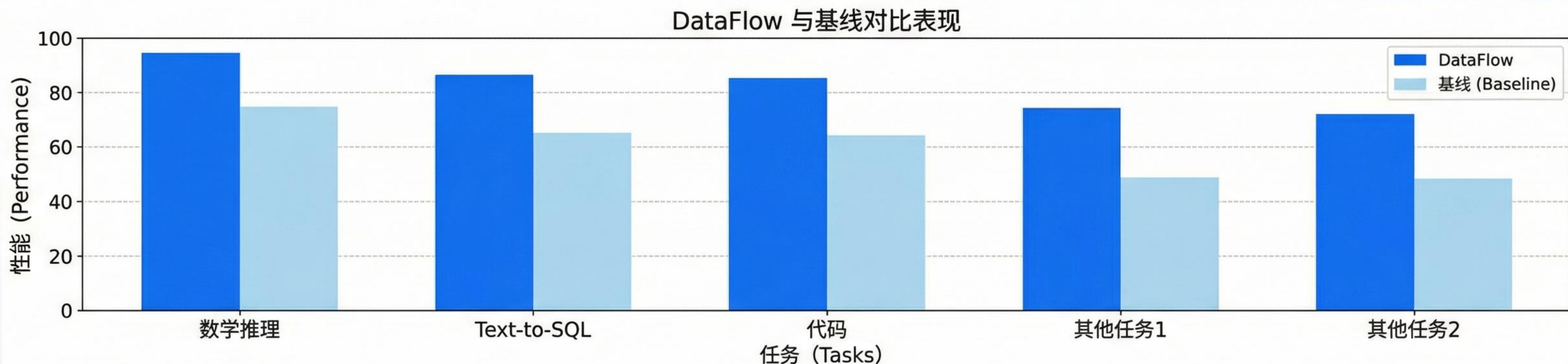
对比基线	类型	示例/说明
人类数据	高质量人工整理	精选数据集，作为黄金标准
合成基线	专用合成流程	特定任务的自动生成数据
现有系统	强基线模型	Qwen2.5-Instruct 系列等

关键评估指标

- 执行准确率 (Text-to-SQL)
- 代码基准平均提升
- 数学基准得分提升
- 统一数据规模与训练配置，对比数据效率与下游任务表现

关键实验结果与性能提升

- **数学推理数据**: 在 MATH、GSM8K、AIME 上相比高质量合成基线提升约 1–3 分。
- **Text-to-SQL**: 在不到 0.1M 样本下, 相比 2.5M SynSQL 语料提升超过 +3% 执行准确率。
- **代码数据**: 相较广泛使用的公共 code instruction 数据集, 平均提升超过 7%。
- **统一数据集 DataFlowInstruct-10K**: 仅 10K 样本即可使 Qwen2/Qwen2.5-base 超过使用 1M Infinity-Instruct 训练的模型。
- **总体结论**: 统一的 LLM 驱动数据准备可在显著提升数据质量的同时, 大幅提高数据效率。



总结与致谢

本文贡献总结

- 提出 DataFlow: 以 LLM 为核心的一体化数据准备框架, 提供统一抽象与高性能执行。
- 构建丰富的运算符与管线生态, 通过 PyTorch 风格 API 与扩展机制降低开发与共享门槛。
- 引入 DataFlow-Agent, 实现从自然语言到可执行管线的自动化编排与调试。
- 在文本、数学、代码、Text-to-SQL、RAG、知识抽取等多任务上获得全面、稳定的性能提升, 显著提高数据效率。

致谢

衷心感谢所有作者团队、合作单位以及开源社区对本工作的宝贵支持与贡献。